

```
var a = 1; let b = 2; const c = 3;
```

```
{ console.log("Inside block"); var a = 10; let b = 20; const c = 30; d = 40;
```

```
  console.log("a = ",a);  
  console.log("b = ",b);  
  console.log("c = ",c);  
  console.log("d = ",d);
```

```
}
```

```
console.log("Outside block");  
  
console.log("a = ",a);  
console.log("b = ",b);  
console.log("c = ",c);  
console.log("d = ",d);
```

Explanation of above code according to GEC.

- GEC will run this code in two phase.
 1. Variable phase.
 2. Execution phase.
- on 5th line variable phase will firstly search for var type variable then he search same var_name will declared outside also or not if happen then it will create a space or otherwise point the same data.
- and in the variable phase they store undefiend for the var and the keep empty for both let and const.
- in the execution phase value of the variable will be assigned to the variable and allocates to the memory.
- In the execution phase when they enter into the block scope then firstly search for the let and const type of variable then hr creates a block scope

otherwise they only update the value of var in the global scope.

- and if they found let and const in block scope then they again run the variable phase for the block scope only.
- and then block scope will be create and will keep empty because they dont assign any value in variable phase.
- After this execution phase will run for the entire code from the where he stopped.
- In the execution phase when we try to assign the value then they firstly they search that variable in own scope(BLOCK SCOPE), and if they dont found then he search in own parent(script scope) and if they dont found also then he goes to global scope and update the value of that variable. (This is called **Lexical scope**)
- In the execution phase, when js engine will exit from block scope then they clear the block scope memory using by using garbage collector features.